

SysMD Language Reference Manual

Christoph Grimm, Sebastian Post, NN, NN

TU Kaiserslautern

Chair of Cyber-Physical Systems

1. Introduction	3
1.1. Context	3
1.2. Language Design Objectives	3
2. Metamodel	5
2.1. Elements	5
2.2. Values as Features	6
2.3. Relationships and annotations	6
2.4. SysMD Profile	7
3. Lexical conventions	8
3.1. Keywords and literals	8
3.2. Operators, pre-defined names	8
3.3. Qualified names	8
3.4. Feature chains	9
4. Syntax and semantics of statements	10
4.1. Triples	10
4.2. Definition of classes	11
4.3. Definition of relationships	12
4.4. Features	12
4.5. Imports	14
5. Semantics of models	16
5.1. Constraint propagation	16
5.2. Inheritance	16
6. Basis projects	17
6.1. Math	17
6.2. Physics	17
6.3. ISO26262	17
6.4. KPI	17

1. Introduction

1.1. Context

For the development of innovative systems, it is crucial to acquire and apply pre-existing knowledge; indeed, knowledge acquisition and its systematic application in an organization has been shown to be a key factor for commercial success of innovations [1]. This is particularly useful in multi-disciplinary environments, where knowledge from various domains must be combined. The challenge there is to avoid the establishment of “silos” or “islands” of knowledge that lack communication and a system-wide overview.

AGILA targets the systematic documentation, flexible retrieval, combination, and systematic application of various kinds of knowledge, including

- Documents in natural language, e.g. as in DOORS; such models have the big advantage that they can be quickly understood by all stakeholders, from different disciplines. However, they lack the precision that is often needed.
- Formalized models, e.g. as in OWL or SysML/UML; such models offer the desired precision, but are not that accessible for all stakeholders and rather target modeling experts.
- Mathematical relations and constraints, e.g. as in OCL; such models are the basis for automated analysis and verification. Such kind of analysis is in particular desirable for agile development and or industrial DevOps scenarios, where the impact of changes must be analyzed often and quickly.

1.2. Language Design Objectives

“Inclusive” modeling process

Requirements elicitation and specification are processes in which the expertise and knowledge of stakeholders from various disciplines must be considered; under an “inclusive” modeling process we understand the fact that tools and languages are such that everybody can be involved easily. SysMD shall support this by the features

1. **Readable model.** The use of a modeling language that allows everybody - even without specific training - to understand them. This means in particular that the language statements shall be close to generally known formalisms like the natural language, basic mathematics, or tables.
2. **Linked description.** A process that begins with a documentation in natural language that is stepwise enriched with more formalized models that however are linked with the documentation and different documents to a living model.
3. **Explicit specification.** The targeted objectives of a specification shall not be mixed with modeling and design artifacts; it must be clear for everybody what is the intention of the specification.

“Living”, non-linear models

The way people communicate has changed.

First, nowadays, communication comes in various formats including figures and pictures, but also more and more multimedia documents like video recordings of discussions. SysMD shall be able to integrate these documents in a single document.

Second, nowadays communication is less linear; there is no longer a clearly defined linear sequence of how to read a document. To speed up communication, and to permit consumption of SysMD models in a more timely way, SysMD shall support

- Classical **linear reading** of a model.
- **Hierarchical reading** of a model, starting with a top-level and then going into details where needed.
- **Relationship-guided reading** of a model, starting with an artifact and following relationships that guide the reader.

Support for classification and semantification

- Linking descriptions and models
- Automatic lookup and use of applicable natural laws or other relationships
- Support by AI for finding or recommending relationships

Consistency Checking

SysMD shall provide expression semantics that support a formal analysis by constraint propagation techniques. The motivation for this lies in more and more agile development processes, where a quick evaluation of the dependencies of elements of a design is needed. Therefore,

Interoperability

SysMD shall focus on the following standards:

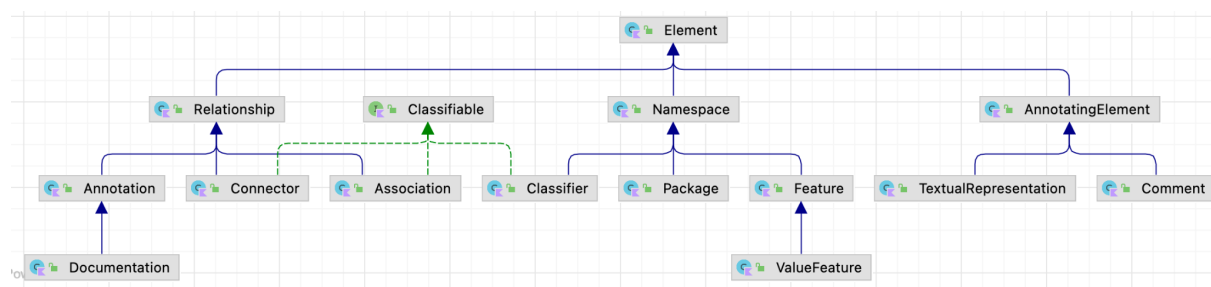
- **Markdown** for documentation. The reason is that this is simple and provides a broad basis for interoperability.
- **SysML v2 KerML subset of the metamodel** for the models that are generated as semantic actions. The reason is that SysML is the de-facto standard for exchange of models, and that from the metamodel one can translate to SysML v2 textual and graphical.
- SysML basic libraries shall provide ontological foundations for interoperability with
 - **ISO 26262 vocabulary**
 - **KPI definitions following ISO ...**

2. Metamodel

The metamodel follows the SysMLv2 standard. However, not all classes of SysMLv2 are supported, and

- `FeatureValue` models the Classes `Feature`, `FeatureValue` and `Expression` (and all subclasses etc. thereof) of SysMLv2.
- `Classifiable` acts as an interface of `Classifier`.
- `Type` is not modeled as it is not instantiated and only needed to describe semantics.
- `Imports`, `FeatureChaining`, `Specialization` of the SysMLv2 metamodel are implemented directly as fields where needed.

The classes of SysMLv2 shown below a class diagram are supported.



2.1. Elements

Element

The class `Element` is the base class of all further classes.

It has the following properties:

- `id`, a UUID4 for unique identification,
- `name`, for identification by name in a given namespace,
- `aliases`, a list of alias-names,
- owned elements,
- owning element.

Namespace

The class `Namespace` is used for searching in a namespace by the name resolution method for the lookup of elements by name.

In addition to the `Element`'s properties it has the property

- `imports`, a list of other `Namespaces`.

Feature

The class `Feature` describes a feature of a class. A `Feature` can be either a `Multiplicity Feature`, i.e. the availability of reified objects of another class in a specified amount.

In addition to a Namespace, a Feature has the following properties:

- `multiplicity`, an integer range,
- `ofClass`, the class (Type) from which the reified objects shall be.

Classifier

The class `Classifier` introduces a super/subclass-relation. In addition to the Namespace, it has the following properties:

- `superclass`, the superclass.

AnnotatingElement

The class `AnnotatingElement` describes complementary information. In addition to an Element, it has the following properties:

- `body`, a String that is the annotation

TextualRepresentation

The class `TextualRepresentation` is an `AnnotatingElement` that is a textual model, i.e. in SysMD or SysMLv2 textual. In addition to an `AnnotatingElement`, it has the following property:

- `language`, a String that represents the language.

2.2. Values as Features

ValueFeature (Property)

The class `ValueFeature` describes a value of specified Type of the Value. It unites several subclasses of SysMLv2 in a single class (`Feature`, `ValueFeature`, `Expression`). Its multiplicity is considered to be 1.

In addition to `Feature`, a `ValueFeature` has the following properties:

- `valueSpec`, a Constraint that specifies constraints to the desired value.
- `unitSpec`, a Unit in which the result is displayed or computed.

2.3. Relationships and annotations

A relationship is an element that models a relationship between elements. In addition to an Element, it has the properties

- `sources`
- `targets`

Annotation

- Relationship between `AnnotatingElement` and an annotated element

Association

SysMD uses the Association class to model all relationships specified by the user. This allows SysMD to classify every relationship.

Connector

(as Association)

2.4. SysMD Profile

In the SysMD Profile we describe deviations from the “Standard” SysML/KerML metamodel and describe how they can be represented by the Standard metamodel.

ValueFeature extension

Objective of the extension is to separate the specification from the computed results of constraint propagation, and the models for computation and constraint propagation. For this reason, for each Feature we represent

- ValueSpec: the specified values as a range or set.
- UnitSpec: the unit in which the value is given or specified (optional)
- Dependency expression: an expression that gives the dependency of the value/unit from other expressions.
- The semantics of IN/OUT is mapped to the SysMD Keywords ALL resp. ONE.

All of the above information is modeled in the SysMD metamodel in one class, whereas SysMLv2 metamodel uses the classes Feature (for the Value, Multiplity), Expression for the expression, and FeatureValue, an Relationship, that establishes a link between value and expression.

Reasons for the deviation: make specification vs. design more explicit and traceable, and efficiency.

GeneratedCode annotations

Basically, within the standard, but quite extensively used are annotations that link the natural-language descriptions with the related code. These relationships link:

- A TextualRepresentation element with the metamodel elements generated from the TextualRepresentation element.
- Including additional information for error tracking

3. Lexical conventions

Lexical conventions shall follow the SysML v2 standard if not specified otherwise.

3.1. Keywords and literals

Keywords are:

- “isA”, “hasA”, “defines”, “imports”, “dependsOn”,
- “all”, “one”, “true”, “false”, “and”, “or”
- MAYBE: “Project”, “Global”, “Any” → optional?
- MAYBE: “Package”, “Instance”, “Class”, “Component”, “Value”, “Function”

Literals are (following the definitions given by SysMLv2 Textual):

- NAME
- INTEGER
- FLOAT
- STRING
- COMMENT

3.2. Operators, pre-defined names

Operators in expressions, as in SysML v2:

- Arithmetic operations ‘+’, ‘-’, ‘*’, ‘/’, ‘^’ (to the power of)
- Comparison operators ‘<’, ‘>’, ‘<=’, ‘>=’, ‘==’
- Assignment operator ‘=’
- Range operator ‘..’

In names, the following operators are allowed:

- Scope operator ‘::’, Feature chain operator ‘.’

In semantic triples, comma (‘,’), semicolon (‘;’), and dot (‘.’) separate enumerated elements.

Pre-defined names are:

- ‘Global’, for a global scope package that owns everything.
- ‘Any’, for a generic class from which everything inherits.

3.3. Qualified names

Qualified names specify a relative path from

1. the elements owned by a namespace, including inherited elements,
2. the imported elements by a namespace,
3. the elements owned by the owner of a namespace.

Example.

`X::Y::Z`

means an Element with name “X” that owns, inherits, or imports an element “Y” that owns, inherits, or imports an element “Z”.

3.4. Feature chains

Extended names specify a path into features, going from one feature to another of a namespace.

Example:

`X.y.z`

means an Element with name “X” that has a Feature with name “y” that has a Feature with name “z”.

(NOT YET IMPLEMENTED)

4. Syntax and semantics of statements

The overall language design follows the concept of semantic triples of the form

Subject Predicate Object .

We allow the `Object` to be lists of

- *enumerations* of (qualified) names or features that are separated by *commas*, or
- *statements* that are separated by *semicolons*.

Each triple has its own scope defined by the `Subject` and ends with a dot.

4.1. Triples

There are three different kind of triples: definitions, features, and imports:

- *Definitions* introduce facts, i.e. universally true statements that are not subject to any change.
- *Features* of an element introduce properties that can be subject to a later refinement in an instance of an element.
- *Imports* add further elements to the scope (visibility) of an element. The import of an element by its name adds the element to the current scope. The import of a project makes its elements accessible.

Syntax

```
SysMD :- Triple* EOF
```

```
Triple :-  
|      Definitions  
|      Features  
|      Imports
```

Semantics

A SysMD model is a sequence of extended semantic triples that each describe a valid statement that introduces an internal representation by the metamodel. With “Execution” we mean that the internal model is built as intended by the statement.

Regarding the semantics, it holds that

- The order of the Triples is irrelevant.
- In case some information is missing for a statement, but becomes known in a later statement, this is not an error, and the execution is postponed after the execution of a statement that introduces the missing information.
- Repeated execution of the same `ExtendedTriple` shall not change the internal state and is not an error.
- Repeated execution of contradictory `ExtendedTriple` shall overwrite the existing internal state and emit at least a warning.

- There must be no cyclic dependencies of definitions.

Examples

The following two lines 1 and 2 lead to an equivalent internal model:

```
1: A isA B. B isA Any.
2: B isA Any. A isA B.
```

The following two lines are correct:

```
1: A isA Any.
2: A isA Any. // Ok to repeat the same statement.
```

The following two lines shall at least give a warning:

```
1: A isA Any. B isA Any.
2: A isA B. // At least warning.
```

The following two lines are an error:

```
1: A isA B.
2: B isA A.
```

4.2. Definitions

Definitions introduce general truths in the scope of a class:

- classifications of elements, by giving a superclass of the elements.
- values or ranges of values, by assigning it a fixed value
- relationships by using a defined association class or the relationship (see 4.4).

Syntax

```
Definitions :-
    QualifiedName "isA" QualifiedName "."
    [NAME ":" ] QualifiedName QualifiedName QualifiedName "."
|
    QualifiedName "defines" DefinitionList "."

DefinitionList :- Definition (";" Definition)*
Definition :-
    NAME "isA" QualifiedName
{
    [NAME ":" ] NAME QualifiedName QualifiedName
```

Semantics of "isA" Relationships

The definitions consist of single definitions, for which the order shall not be relevant. Each definition introduces an instance of the class `Classifier` where

- Owning element is
 - Specified by the Path of the QualifiedName for a simple definition (without "defines")

- A Namespace specified by the QualifiedName left of “defines”
- Superclass is specified by the QualifiedName after “isA”.
- Name is specified by the name before “isA”.
- It is not allowed to use a QualifiedName as a Subject in a DefinitionList.

Examples for “isA” Relationships

The following statement creates a Classifier with name “A” in the Namespace “Global” and superclass “B”:

```
A isA B.
```

The following statements both create a Classifier with name “B” in the Namespace “A” and superclass “C”:

```
1: A::B isA C.
```

```
2: A defines B isA C.
```

The following statement creates a Classifier “B” in the Namespace “A” with a superclass “B” from the Namespace “C”:

```
A::B isA C::B.
```

Not allowed is for example the following statement. A definition may only define subjects that it owns:

```
A defines B::C isA D.
```

4.3. Definition of relationships

Relationships can have a user-given name and class and are specified by a semantic triple in the definitions section where

- Subject and object must resolve to classes.
- The predicate must resolve to a class that is a `Relationship`.

The definitions may be located before or after the use of a relationship.

Syntax

```
Relationship :-
    QualifiedName AssociationName TargetList “.”
```

Semantics

Use without a definition creates a `Relationship` that is only identified by its name.

If an association was defined via a Classifier (`x isA Relationship`), an `Association` will be created.

Examples

```
Global defines
    hasFunction isA Relationship(Component, Function);
A isA Component;
B isA Function.
```

```
A hasFunction B.
```

TODO: Tests.

4.4. Features

Features specify potential relationships between a class and other classes.

Syntax

```
Features :- QualifiedName "hasA" FeatureList "."

FeatureList :- Feature ("," Feature)*
Feature :- InstanceFeature | ValueFeature
NAME ":" TypeName [ValueSpec] [UnitSpec] ["="
ValueExpression]
NAME ":" [MultiplicitySpec] TypeName ["=" InstanceExpression]
```

Semantics of InstanceFeature

Features are specified by a List of Features following the hasA keyword. The order of the list shall be irrelevant. An InstanceFeature creates an instance of the class Feature with

- OwningElement specified by the QualifiedName as Subject of the Triple.
- Name = NAME.
- MultiplicitySpec = MultiplicitySpec; if not given, MultiplicitySpec = [1, 1].
- ofType is a QualifiedName that resolves to a Classifier.
- (InstanceExpression is an Expression that gives a set of configured instances.)

Example with instanceFeature

```
Car hasA
    engine: Engine,
    wheels: [3..6] Wheels.
```

Semantics of ValueFeature

An instance of ValueFeature is added to the owning elements.

- TypeName must resolve to a class that is a Value.

Example with ValueFeature

```
Car has
    mass: Mass(100 .. 1000) [kg] = bySubclasses(mass),
    power: Power(1 .. 1000) [kW] = engine.power.
```

Semantics of ValueExpression

The ValueExpression can be a calculation with the basic operations like "+", "-", "*", "/" of some Features. Also normal numbers can be included. This numbers can be used in the following ways:

- as normal numbers like the Real value 4.5 or the Integer value 5
- as intervals like [min..max]
- as functions with an interval like Real(min..max) or Integer(min..max). Also other domains like Voltage(min..max) are possible

It is also possible to use the following functions:

- sqrt(x) (square root), sqr(x) (square), ln(x) (natural logarithm), pow(x, y) (x to the power of y), exp(x) (exponential function), max(x, y) (maximum of x and y), min(x, y) (minimum of x and y),
- floor(x) (round down to next Integer), ceil(x) (round up to next Integer)
- sum_i(x, y, a) (sum between x and y, for each step the calculation a, which must include i, is done and summed up)
- sum("x") (sum of all owned elements with the name "x")
- linear(x, x0, y0, x1, y1) (linear interpolation with the value x between (x0,y0) and (x1,y1). If x is smaller than x1 or bigger than x2, y1 or y2 is returned. Also a third point (x2,y2) can be added as a parameter for the function)
- ite(a, y, z) (IF a THEN x ELSE z)
- Date("x"), DateTime("x"), Month("x") (Different Date formats, the "x" must follow the rules of ISO 8601. DateTime contains a date with an exact timestamp, Date only the date without a time and Month a date without a specific day)

Example with ValueFeature

```
Car hasA
    power: Real(0 .. INF) [W],
    f_c: Real(0 .. INF) [Hz] = 1.0 [kHz]/(2*PI),
    speed : Real(0 .. 200) [km/h],
    noise: Real(0 .. INF) [W/Hz] = power/f_c + [0.0 ..
10.0],
    isEnabled: Boolean(true),
    number: Real = sqr(5.0) + Real(-5.0 .. 5.0),
    date: Real(DateTime) = DateTime("2022-31-12T23:59:59").
```

4.5. Imports

Imports can be of two kind:

- 1) Import of another namespace into the given namespace.
- 2) Import of a given project into the current project.

Syntax

```
Imports :-  
    QualifiedName "imports" NamespaceList "."  
|    "Project" "imports" ProjectList "."  
  
NamespaceList :- QualifiedName ("," QualifiedName)*  
ProjectList    :-      Name ("," Name)*
```

Semantics

Importing a Namespace adds the Elements specified by a Qualified name to the scope of the Namespace.

Importing a Project adds all roots of the specified project (default commit) to the current project.

5. Semantics of models

A SysMD Model is a network of elements.

5.1. Constraint propagation

One-Semantics

All-Semantics

5.2. Inheritance

Values

Features

Relationships

6. Basis projects

6.1. Math

Defines math constants:

- pi
- e

6.2. Physics

Defines physical constants

- e0
- m0

6.3. ISO26262

Defines the ISO26262 vocabulary

- Element
- Function
- Component
- HardwarePart, HardwareComponent
- SoftwareUnit, SoftwareComponent

6.4. KPI

Defines a vocabulary of qualities and metrics for modeling performances

- ...